

TNSI — TP1 : banque de sujets ECE

20 sujets numérotés, un par page — à tirer au sort et à imprimer/afficher un par un à l'élève concerné.

Sujets 1 à 10 : variantes modifiées des exercices du TP1 (interrogation standard).
Sujets 11 à 20 : énoncés identiques au TP, pour un tirage d'échauffement ou un élève plus fragile.
Ne remettre à l'élève que la page correspondant au numéro tiré — ne pas partager ce document dans son ensemble.

N°	TYPE	EXERCICE D'ORIGINE	SUJET	PARTIE / N°
1	variante	binaire	<code>octal</code>	Partie 1, n°1
2	variante	distances	<code>point_le_plus_loin</code>	Partie 1, n°2
3	variante	listes (points de rupture)	<code>genes_0</code>	Partie 1, n°3
4	variante	dictionnaires (plan d'envoi)	<code>taille_cycle</code>	Partie 1, n°4
5	variante	listes (max)	<code>minimum</code>	Partie 2, n°1
6	variante	listes (recherche)	<code>compte_occurrences</code>	Partie 2, n°2
7	variante	listes (max et indice)	<code>min_indice</code>	Partie 2, n°3
8	variante	listes (tri)	<code>tri_strict</code>	Partie 2, n°4
9	variante	listes (moyenne)	<code>etendue</code>	Partie 2, n°5
10	variante	chaînes de caractères	<code>correspond_insensible</code>	Partie 2, n°6
11	identique	binaire	<code>binaire</code>	Partie 1, n°1
12	identique	distances	<code>distances</code>	Partie 1, n°2
13	identique	listes (points de rupture)	<code>listes</code>	Partie 1, n°3
14	identique	dictionnaires (plan d'envoi)	<code>dictionnaires</code>	Partie 1, n°4
15	identique	listes (max)	<code>max</code>	Partie 2, n°1
16	identique	listes (recherche)	<code>recherche</code>	Partie 2, n°2
17	identique	listes (max et indice)	<code>max_indice</code>	Partie 2, n°3
18	identique	listes (tri)	<code>tri</code>	Partie 2, n°4
19	identique	listes (moyenne)	<code>moyenne</code>	Partie 2, n°5
20	identique	chaînes de caractères	<code>trous_chaines</code>	Partie 2, n°6

octal

D'après l'exercice « binaire » (Partie 1, n°1 du TP1)

Même principe, mais en base 8. Écrire `octal(a)` qui prend en paramètre un entier positif `a` et renvoie une chaîne de caractères contenant son écriture en base 8, obtenue par divisions euclidiennes successives par 8.

CODE À COMPLÉTER

```
def octal(a):  
    """Convertit un nombre entier a en sa représentation en base 8 (chaîne de  
    caractères)."""  
    if a == 0:  
        return ...  
    oct_a = ...  
    while ... :  
        oct_a = ... + oct_a  
        a = ...  
    return oct_a
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

point_le_plus_loin

D'après l'exercice « distances » (Partie 1, n°2 du TP1)

On garde `distance_carre(point1, point2)` à écrire comme dans l'exercice original. On souhaite maintenant trouver le point le plus **éloigné** du point de départ. Écrire `point_le_plus_loin(depart, tab)` sur le même principe (en cas d'égalité, on garde le premier point trouvé).

CODE À COMPLÉTER

```
def distance_carre(point1, point2):
    """ Calcule et renvoie la distance au carré entre deux points."""
    return (...)**2 + (...)**2

def point_le_plus_loin(depart, tab):
    """ Renvoie les coordonnées du premier point du tableau tab se trouvant à
    la plus GRANDE distance du point depart."""
    max_point = tab[0]
    max_dist = ...
    for i in range(1, len(tab)):
        if distance_carre(tab[i], depart) > ...:
            max_point = ...
            max_dist = ...
    return max_point
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (0-indexées)

D'après l'exercice « listes (points de rupture) » (Partie 1, n°3 du TP1)

Modification : les gènes sont maintenant numérotés de **0 à n-1** (au lieu de 1 à n). Le principe est le même que dans l'exercice original : il y a un point de rupture si la première valeur n'est pas 0, si l'écart entre deux gènes consécutifs n'est pas égal à 1, ou si la dernière valeur n'est pas n-1. Compléter `est_un_ordre_0(tab)` et `nombre_points_rupture_0(ordre)` sur ce principe.

CODE À COMPLÉTER

```
def est_un_ordre_0(tab):
    ''' Renvoie True si tab est de longueur n et contient tous les entiers de 0
    à n-1, False sinon '''
    n = len(tab)
    vus = ...
    for x in tab:
        if x < ... or x > ... or ... :
            return False
        ....append(...)
    return True

def nombre_points_rupture_0(ordre):
    ''' Renvoie le nombre de points de rupture de ordre (gènes numérotés de 0 à
    n-1) '''
    assert ...
    n = len(ordre)
    nb = 0
    if ordre[...] != 0:
        nb = nb + 1
    i = 0
    while i < ... :
        if ... not in [-1, 1]:
            nb = nb + 1
        i = i + 1
    if ordre[i] != ... :
        nb = nb + 1
    return nb
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

taille_cycle

D'après l'exercice « dictionnaires (plan d'envoi) » (Partie 1, n°4 du TP1)

On ne part plus forcément de 'A' : écrire `taille_cycle(plan, personne)` qui renvoie la taille (le nombre de personnes) du cycle auquel appartient `personne` dans le plan d'envoi `plan`, en suivant le même principe que `est_cyclique` mais en partant de `personne` plutôt que de 'A', et en renvoyant le nombre de personnes du cycle plutôt qu'un booléen.

CODE À COMPLÉTER

```
def taille_cycle(plan, personne):  
    '''Renvoie le nombre de personnes dans le cycle contenant personne.'''  
    expéditeur = ...  
    destinataire = plan[...]  
    taille = 1  
    while destinataire != ... :  
        destinataire = ...  
        taille = ...  
    return taille
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

minimum_tableau

D'après l'exercice « listes (max) » (Partie 2, n°1 du TP1)

Écrire `minimum_tableau(tab)`, qui prend en paramètre un tableau non vide de nombres `tab` et renvoie le plus **petit** élément de ce tableau. L'utilisation de la fonction native `min` n'est pas autorisée.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

compte_occurrences

D'après l'exercice « listes (recherche) » (Partie 2, n°2 du TP1)

Écrire `compte_occurrences(tab, n)`, qui prend en paramètre un tableau non vide d'entiers `tab` et un entier `n`, et renvoie le **nombre de fois** où `n` apparaît dans `tab` (0 si absent).

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

min_et_indice

D'après l'exercice « listes (max et indice) » (Partie 2, n°3 du TP1)

Écrire `min_et_indice(tab)`, qui prend en paramètre un tableau non vide `tab` de nombres et renvoie un tuple `(valeur_min, indice)` avec la valeur du plus petit élément et l'indice de sa **première** apparition. L'utilisation de la fonction native `min` n'est pas autorisée.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

verifie_strict

D'après l'exercice « listes (tri) » (Partie 2, n°4 du TP1)

Écrire `verifie_strict(tab)`, qui prend en paramètre un tableau de valeurs numériques et renvoie `True` si ce tableau est trié dans l'ordre **strictement** croissant (deux éléments consécutifs égaux invalident le tri), `False` sinon. Un tableau vide ou à un élément est considéré comme trié.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

etendue

D'après l'exercice « listes (moyenne) » (Partie 2, n°5 du TP1)

Écrire `etendue(tab)`, qui prend en paramètre un tableau non vide de nombres flottants et renvoie son **étendue**, c'est-à-dire l'écart entre le plus grand et le plus petit élément du tableau (valeur maximale moins valeur minimale).

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

correspond_insensible

D'après l'exercice « chaînes de caractères » (Partie 2, n°6 du TP1)

Même principe, mais la comparaison doit être **insensible à la casse** : `mot` peut être écrit en minuscules, majuscules ou mélangé, et doit quand même correspondre au mot à trous (toujours en majuscules). Écrire `correspond_insensible(mot, mot_a_trous)`.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

binaire

Exercice original (Partie 1, n°1 du TP1)

Écrire `binaire(a)` : prend en paramètre un entier positif `a` et renvoie une chaîne de caractères contenant son écriture en base 2, obtenue par divisions euclidiennes successives par 2 (le premier reste calculé est le bit de poids faible).

CODE À COMPLÉTER

```
def binaire(a):  
    """Convertit un nombre entier a en sa représentation binaire (chaîne de  
    caractères)."""  
    if a == 0:  
        return ...  
    bin_a = ...  
    while ... :  
        bin_a = ... + bin_a  
        a = ...  
    return bin_a
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

distances

Exercice original (Partie 1, n°2 du TP1)

On rappelle que la distance d entre deux points $(x;y)$ et $(x';y')$ vérifie $d^2 = (x-x')^2 + (y-y')^2$. Compléter `distance_carre(point1, point2)` (les points sont des tuples (x, y)) et `point_le_plus_proche(depart, tab)`, qui renvoie les coordonnées du premier point du tableau `tab` se trouvant à la plus courte distance du point `depart`.

CODE À COMPLÉTER

```
def distance_carre(point1, point2):
    """ Calcule et renvoie la distance au carré entre deux points."""
    return (...)**2 + (...)**2

def point_le_plus_proche(depart, tab):
    """ Renvoie les coordonnées du premier point du tableau tab se trouvant à
    la plus courte distance du point depart."""
    min_point = tab[0]
    min_dist = ...
    for i in range(1, len(tab)):
        if distance_carre(tab[i], depart) < ...:
            min_point = ...
            min_dist = ...
    return min_point
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (points de rupture)

Exercice original (Partie 1, n°3 du TP1)

Un tableau `ordre` de longueur `n` contient des entiers distincts entre 1 et `n` : c'est un ordre de gènes de chromosome. Il y a un **point de rupture** si la première valeur n'est pas 1, si l'écart entre deux gènes consécutifs n'est pas égal à 1, ou si la dernière valeur n'est pas `n`. Compléter `est_un_ordre(tab)` (teste si `tab` est bien un ordre valide) et `nombre_points_rupture(ordre)` (compte les points de rupture).

CODE À COMPLÉTER

```
def est_un_ordre(tab):
    ''' Renvoie True si tab est de longueur n et contient tous les entiers de 1
    à n, False sinon '''
    n = len(tab)
    vus = ...
    for x in tab:
        if x < ... or x > ... or ... :
            return False
        ....append(...)
    return True

def nombre_points_rupture(ordre):
    ''' Renvoie le nombre de points de rupture de ordre qui représente un ordre
    de gènes de chromosome '''
    assert ...
    n = len(ordre)
    nb = 0
    if ordre[...] != 1:
        nb = nb + 1
    i = 0
    while i < ... :
        if ... not in [-1, 1]:
            nb = nb + 1
        i = i + 1
    if ordre[i] != ... :
        nb = nb + 1
    return nb
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

dictionnaires (plan d'envoi)

Exercice original (Partie 1, n°4 du TP1)

Un « plan d'envoi » est un dictionnaire du type `{'A':'E', 'B':'F', ...}` où chaque personne envoie des messages à une seule personne (destinataire). Un **cycle** est une suite de personnes qui reboucle sur elle-même. Le plan est **cyclique** s'il ne comporte qu'un seul cycle qui couvre tout le monde. Compléter `est_cyclique(plan)` : en partant de `'A'`, on suit les destinataires jusqu'à retomber sur `'A'`, et on vérifie que tout le monde a été visité.

CODE À COMPLÉTER

```
def est_cyclique(plan):  
    '''Renvoie True si le plan d'envoi de messages est cyclique et False sinon.'''  
    expéditeur = 'A'  
    destinataire = plan[...]  
    nb_destinataires = 1  
    while destinataire != expéditeur:  
        destinataire = ...  
        nb_destinataires = ...  
    return nb_destinataires == ...
```

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (max)

Exercice original (Partie 2, n°1 du TP1)

Écrire `maximum_tableau(tab)`, qui prend en paramètre un tableau non vide de nombres `tab` et renvoie le plus grand élément de ce tableau. (*Code complet à écrire soi-même, avec commentaires.*)

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (recherche)

Exercice original (Partie 2, n°2 du TP1)

Écrire `recherche(tab, n)`, qui prend en paramètre un tableau non vide d'entiers `tab` et un entier `n`, et renvoie l'indice de la **dernière** occurrence de `n` dans `tab`. Si `n` n'est pas présent, la fonction renvoie `None`.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (max et indice)

Exercice original (Partie 2, n°3 du TP1)

Écrire `max_et_indice(tab)`, qui prend en paramètre un tableau non vide `tab` de nombres et renvoie un tuple `(valeur_max, indice)` avec la valeur du plus grand élément et l'indice de sa **première** apparition. L'utilisation de la fonction native `max` n'est pas autorisée.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (tri)

Exercice original (Partie 2, n°4 du TP1)

Écrire `verifie(tab)`, qui prend en paramètre un tableau de valeurs numériques et renvoie `True` si ce tableau est trié dans l'ordre croissant (*au sens large*, égalités autorisées), `False` sinon. Un tableau vide ou à un élément est considéré comme trié.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

listes (moyenne)

Exercice original (Partie 2, n°5 du TP1)

Écrire `moyenne(tab)`, qui prend en paramètre un tableau non vide de nombres flottants et renvoie la moyenne des valeurs du tableau.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.

chaînes de caractères

Exercice original (Partie 2, n°6 du TP1)

On considère des « mots à trous » : chaînes ne contenant que des majuscules et des `*` (ex. `INFO*MA*IQUE`). Écrire `correspond(mot, mot_a_trous)` qui renvoie `True` si on peut obtenir `mot` en remplaçant convenablement les `*` de `mot_a_trous`, `False` sinon.

BROUILLON / ANALYSE

Interrogation individuelle — 20 minutes — sans accès Internet, sans IA. Le code s'écrit sur le poste de travail ; cette page ne sert qu'à l'énoncé.